sentiment analysis. In this setup, we define a sentiment concept S and use a corresponding sentiment analysis dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{n}$, where y_i represents the labeled sentiment of sentence x_i. Here, $\text{rep}_\theta(x_i)$ represents an intermediate representation of x_i, generated from a forward pass through the LLM_θ. The goal is to extract a collection of these representations $\{\text{rep}_\theta(x_i)\}_{i=1}^{n}$. We therefore train a classifier C_w on these representations to predict the presence of the sentiment concept S (i.e., positive or negative sentiment within x_i), effectively predicting y_i.

Implementing sentiment detection involves two fundamental design choices: (1) The selection of the intermediate representation rep_θ, which may vary depending on the layers or pooling strategies applied within the LLM. (2) The choice of classifier C_w, which serves to distinguish between different sentiment categories based on these embeddings.

In the following, we outline the fundamental structure of Transformer architecture and highlight common techniques for selecting representations.

Choice of Representation. Current state-of-the-art LLMs are based on the Transformer architecture (Vaswani et al., 2017), where sequential Transformer blocks generate intermediate hidden representations (h), each with potentially distinct functionalities. Let $l \in \mathbb{N}$ denote the l-th layer, and $\mathbf{x}^{(l)} \in \mathbb{R}^{T \times d_{\text{emb}}}$ represent the output, where T is the number of tokens and d_{emb} is the hidden dimension. A Transformer refines $\mathbf{x}^{(l)}$ using multi-head attention (MHA) and a feed-forward network (FFN):

$$h_{\text{attn}}^{(l)} = \text{MHA}\left(\text{LayerNorm}\left(x^{(l)}\right)\right)$$

$$h_{\text{resid}}^{(l)} = h_{\text{attn}}^{(l)} + x^{(l)}$$

$$h_{\text{ffn}}^{(l)} = \text{FFN}\left(\text{LayerNorm}\left(h_{\text{resid}}^{(l)}\right)\right)$$

$$x^{(l+1)} = h_{\text{ffn}}^{(l)} + h_{\text{resid}}^{(l)}$$

We exploit the Llama-3 architecture (Dubey et al., 2024), leveraging its Grouped Query Attention (GQA) and RMSNorm features to extract hidden representations efficiently.

Previous studies have explored various representations, such as the residual stream ($\mathbf{x}^{(l+1)}$) (Marks and Tegmark, 2023; Burns et al., 2023; Zou et al., 2023; Gurnee and Tegmark, 2024), the normalized residual stream (nostalgebraist, 2020), or attention heads (Li et al., 2023; Arditi and Obeso, 2023). Based on our preliminary experiments showing marginally higher detection accuracy, we adopt the residuals stream ($\text{rep}_\theta(\mathbf{x}) = \mathbf{x}^{(l+1)}$).

Instead of using the full prompt representation ($\mathbf{x}_{\text{rep}} \in \mathbb{R}^{T \times d_{\text{emb}}}$, where T is the token count), we focus on a subset ($\mathbf{x}_{\text{rep}} \in \mathbb{R}^{t \times d_{\text{emb}}}$) with $t \leq T$. Each token representation ($\mathbf{x}_{\text{rep}}[i, :] \in \mathbb{R}^{d_{\text{emb}}}$, for $i = 1, \ldots, t$), is treated as an independent feature. This approach focuses on the parts of the prompt most likely to capture the sentiment concept. Prior work has carefully selected a single token (Arditi and Obeso, 2023; Zou et al., 2023; Gurnee and Tegmark, 2024) or relied on the last token of the prompt (Rimsky, 2023; Mallen and Belrose, 2023; Marks and Tegmark, 2023; Li et al., 2023; Burns et al., 2023).

In our experiments, we explore six methods for selecting the representations:

(1) Mean Pooling (Fig. 2): Compute the mean activation value across all tokens for each d_{embed} dimension, resulting in a single vector where each element corresponds to the average activation of a particular feature (embedding dimension) over the entire sequence.

(2) Last-Token Pooling (Fig. 3): Uses the final token's features of the last token in the sequence.

(3) Max Pooling (Fig. 4): Identifies the most prominent feature across all tokens T for each feature, outputting a vector representing the most dominant features in the sequence.

(4) Min Pooling (Fig. 4): Complementary to Max Pooling, producing a vector representing the least dominant features in the sequence.

(5) Concat-Mean-Max-Min Pooling: Concatenates the mean, max, and min pooling, generating a representation of size $3 \times d_{\text{embed}}$ that encapsulates multiple aspects of the token embeddings.

(6) Attention Mean Pooling (Fig. 5): Given a token representation $\mathbf{x}_{\text{rep}}[i, j]$, where i denotes the token and j represents a specific embedding dimension, this pooling method constructs a representation by combining token embeddings with a corresponding importance score. The importance score for each token i is computed by applying the softmax function to the mean of the token's embedding values. This assigns higher weights to tokens with larger average values. These scores are multiplied element-wise to each dimension of the token embeddings, $x_{\text{rep}}[i, :]$, to adjust their contribution based on importance. The final pooled

representation, x_{pooled}, is then computed as a weighted sum of these adjusted embeddings, emphasizing the most relevant tokens activations. Mathematically:

$$x_{\text{pooled}} = \sum_{i=1}^{T} \text{softmax}\left(\frac{1}{J}\sum_{j=1}^{J} x_{\text{rep}}[i,j]\right) \cdot x_{\text{rep}}[i,:]$$

A visual representation of the pooling strategies can be found in Appendix D. These approaches offer diverse strategies for selecting representations, enabling a more nuanced understanding of how sentiment information is encoded. The choice of tokens should ideally reveal whether sentiment is distributed across the entire prompt or concentrated in specific tokens. For instance, the sentences "My name is XYZ and I'm happy" and "I'm happy because my name is XYZ" both convey positive sentiment. However, relying solely on the last token could lead to a suboptimal representation, as it may not adequately capture the sentiment expressed earlier in the sequence.

Choice of Classifier. With the hidden representations selected, we can train our probing classifier C_w on the sentiment labels. In our experimental setup we include twelve distinct classifiers, grouped into five categories based on their underlying modeling approach, as summarized in Table 1.

Type	Models
Linear	Logistic Regression, Linear SVM
Distance-based	K-Nearest Neighbors
Tree-based	Decision Tree, Random Forest, XGBoost, LightGBM
Neural Network	MLP, BiLSTM, CNN
Other	Non-linear SVM, Gaussian Naive Bayes

Table 1: Clusters of Classifiers Based on Model Type

These classifiers were chosen to balance simplicity, interpretability, and the capacity to model complex patterns. Linear models, such as Logistic Regression and Linear SVM, serve as baselines for probing tasks due to their simplicity and their ability to identifying and leveraging linear relationships. Non-linear and neural network models, on the other hand, are included for their ability to capture intricate relationships within the hidden space. BiLSTM and CNN, in particular, were selected for their proven ability to generate effective hidden representations (Ghafoor et al., 2023).

All models, except for BiLSTM and CNN, are implemented using the scikit-learn library (Pedregosa et al., 2011). While, BiLSTM and CNN were implemented using PyTorch. For each classifier, we employed the Optuna framework (Akiba et al., 2019) for hyperparameter optimization, focusing on key parameters such as regularization strengths, tree depths, and kernel types. Optimization was performed over five trials (Paparella et al., 2023), balancing computational efficiency with sufficient exploration of the hyperparameter space.

To ensure reproducibility, we configured random seed initialization and enforced deterministic behavior for CUDA operations. Details are provided in Appendix B.

Datasets Details. Our experiments utilize three benchmark datasets for sentiment classification: IMDB (Maas et al., 2011), SST-2 (Socher et al., 2013), and Rotten Tomatoes (Pang and Lee, 2005), all of which contain movie reviews for binary polarity tasks (e.g., positive or negative sentiment). Additionally, we extended the setup to a more complex evaluation, conducting fine-grained sentiment classification using the Emotion dataset (Saravia et al., 2018), which categorizes sentiments into six nuanced classes: joy, sadness, anger, fear, love, and surprise. We only preprocessed the IMDB and Emotion datasets because their original sizes, 50K for IMDB and 20K for Emotion, made them impractical for the large number of experiments. Details of the preprocessing steps are provided in Appendix A. Table 2 provides details on the number of samples and the train-test splits used in the experiments.

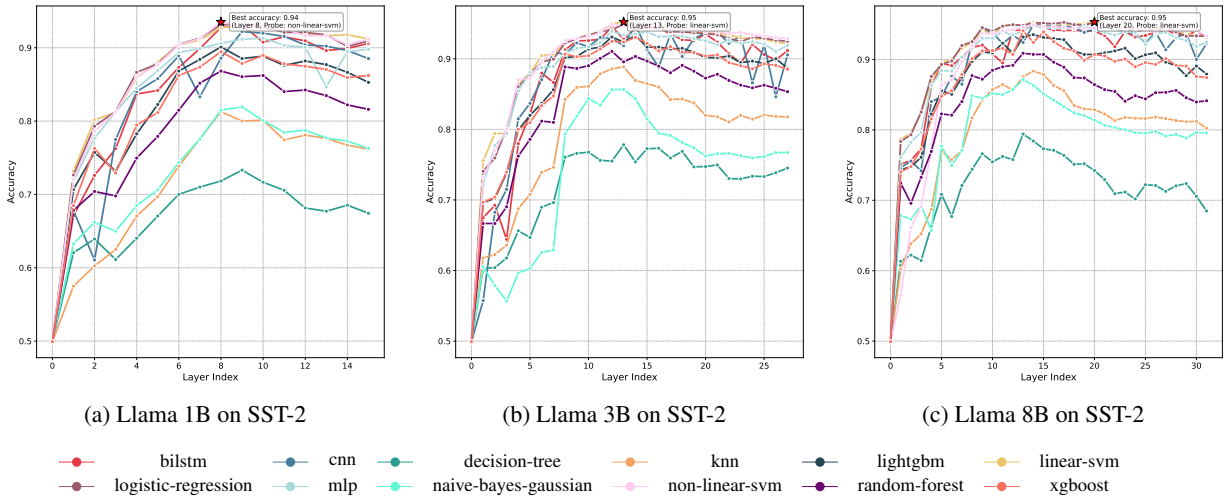
Dataset	Train	Test	Labels	Max Len
IMDB	7000	7000	pos/neg	132
SST-2	6920	1821	pos/neg	56
Rotten Tomatoes	8530	1066	pos/neg	59
Emotion	6000	2000	six labels	64

Table 2: Dataset statistics.

4 Experimental Results

We conduct our experiments on three Llama models, specifically testing the instruction-tuned variants of Llama 3.2-1B, Llama 3.2-3B, and Llama 3.1-8B, all sourced from Hugging Face[2]. For every

[2] https://huggingface.co/meta-llama



(a) Llama 1B on SST-2 (b) Llama 3B on SST-2 (c) Llama 8B on SST-2

Figure 1: Layer-wise probing accuracy using the Last-Token approach on SST-2; Appendix E for additional dataset results.

probing datasets we train the classifier C_w on the training set, and evaluate its detection performance by measuring the accuracy on the test set.

Sentiment detection results. In our first experiments, we evaluate the detection performance of the different classifier C_w. To extract the rep_θ, following prior work, we focus on the residual stream using the Last-Token approach. We report detection accuracies for each layer in LLM_θ and visualize the results in Fig. 1.

For sentiment detection in binary polarity tasks (i.e., SST-2, IMDB, and Rotten Tomatoes), non-linear SVM, linear SVM, and logistic regression consistently outperform other probing techniques, achieving approximately 90% accuracy in the middle layers across all model sizes. For fine-grained emotion datasets, linear SVM demonstrates the best performance, reaching around 70% accuracy in the initial layers across all model sizes. This indicates that Llama models have linear representations for binary sentiment (positive/negative) and fine-grained emotions (joy, sadness, anger, fear, love, and surprise). It is interesting to discover that, although sentiment is not easily discernible during direct interaction with Llama, sentiment and emotion concepts can be linearly detected within certain internal layers.

Furthermore, since prior work often use the last-token representations, we also investigate the effect of different token representation methods. Specifically, we compare the last-token approach with five alternatives: mean, max, min, concatenation, and attention, as previously described.

The results, summarized in Table 3, which shows the top-3 performers by datasets, reveal that concatenating the mean, max, and min of rep_θ is often the most effective method for detecting sentiment concepts. Notably, mean and attention pooling also perform strongly, achieving results comparable to concatenation and consistently outperforming last-token pooling. This indicates, that the last-token representation is not always the optimal choice for capturing sentiment concepts.

Furthermore, we discover that combining the max pooling representation with LightGBM yields particularly strong performance. Further investigation reveals that tree-based models, including Random Forest, Decision Tree, and XGBoost, also perform better with max pooling. We hypothesize that the synergy between max pooling and tree-based models arises from max pooling's ability to emphasize dominant features, which, when combined with tree-based models' strength in exploiting high-contrast, threshold-based partitions, leads to an optimal combination. This alignment makes max pooling particularly effective for tree-based models, highlighting how the choice of classifier should also influence the choice of representation, and vice versa.

5 SENTRILLAMA for Efficient Downstream Tasks

Sentiment analysis has been a fundamental task since the inception of NLP. Traditionally, there have been two primary approaches to accomplish this task: (1) training a model from scratch or (2)